

目录

Tricky	1
最小生成树	2
最短路	2
匹配问题	2
保证完备匹配时	4
不保证完备匹配时	4
2-Sat	5
模型	5
判定	5
构图方法	5
Tarjan	6
点双连通分量对边染色	6
点双连通分量对点染色	6
边双连通分量	7
强连通分量	8
Max Flow	8
Min Cost Max Flow	10
Spfa	10
Dijkstra (没有负圈的情况下)	12
Flow Tricky	13
特定流量	13
分析最小割	14
最大权闭合子图	14
最大密度子图	14
无源汇上下界可行流	15
有源汇上下界可行流	15
有源汇上下界最大流	15
有源汇上下界最小流	16
费用流	16
Match	16
二分图匹配	16
二分图最大权完美匹配	18
一般图匹配	20
Count Circle Number	22

Tricky

1. 当某个图的关系不好辨别时, 可以把它变成它的反图或补图, 从而解决问题.
2. 平面图边数是 $O(n)$ 量级的
3. 搜索树
 - 无向图: 树边, 非树边 有向图: 树边, 正向边, 反向边, 横叉边.
 - 横叉边和非树边的重要性质: $Dfn_y < Dfn_x$, 已经访问, 且包含了信息.
4. 利用 floyd 的转移模式做题

5. 利用并查集维护信息，关于集合或者连通性
6. 最小割树？
7. 树上哈希算法

这类方法需要一个多重集的哈希函数。以某个结点为根的子树的哈希值，就是以它的所有儿子为根的子树的哈希值构成的多重集的哈希值，即：

$$h = f(\{h_i \mid i \in \text{son}(c)\})$$

其中 h 表示以 c 为根的子树的哈希值， f 是多重集的哈希函数。

以代码中使用的哈希函数为例：

$$f(S) = \left(\sum_{c \in S} g(c) \right) \bmod m$$

其中 c 为常数，一般使用 1 即可。 m 为模数，一般使用 2^{32} 或 2^{64} 进行自然溢出，也可使用大素数。 g 为整数到整数的映射，代码中使用 xor shift，也可以选用其他的函数，但是不建议使用多项式。为了预防出题人对着 xor hash 卡，还可以在映射前后异或一个随机常数。

这种哈希十分好写。如果需要换根，第二次 DP 时只需把子树哈希减掉即可。

8. 差分约束算法

判断无解是可以通过题目性质减少约束次数的。

9. $O(n \log n) - O(1)$ LCA

稀疏表的下表是 dfn ，代表 dfs 序区间内深度最小的点的父亲，每次询问若相等直接返回 x ，否则返回 $[dfn_x + 1, dfn_y]$ 的答案。

10. Floyd+bitset 传递闭包

最小生成树

1. Kruskal 重构树
2. Boruvka 算法

最短路

1. Johnson 全源最短路

新建一个虚拟节点（设其编号为 0）。从这个点向其他所有点连一条边权为 0 的边。

Bellman-Ford 算法求出从 0 号点到其他所有点的最短路，记为 h_i 。

假如存在一条从 u 点到 v 点，边权为 w 的边，则我们将该边的边权重新设置为 $w + h_u - h_v$ 。

2. 同余最短路

匹配问题

1. 匈牙利算法时间复杂度 $O(m \times \min\{n_L, n_R\})$ ，Dinic 算法时间复杂度 $O(m\sqrt{\min\{n_L, n_R\}})$
2. 二分图匹配中，按边排序可以使左部点匹配的右部点，优先匹配最前的边。

3. Berge 引理

对于图 $G = (V, E)$ 和它的一个匹配 M , 匹配 M 是最大匹配, 当且仅当不存在相对于匹配 M 的增广路.

4. Hall 定理

假设 $G = (X, Y, E)$ 是二分图, 且 $|X| \leq |Y|$. 对于任何 $W \subseteq X$, 记 $N_G(W)$ 为图 G 中所有与 W 中的顶点相邻的顶点集合. 那么, X - 完美匹配存在, 当且仅当 $|W| \leq |N_G(W)|$, 对于所有 $W \subseteq X$ 都成立.

5. König 定理

二分图中, 最小点覆盖中的顶点数量等于最大匹配中的边数量.

点覆盖相当于选出一个点集合, 使得所有边都存在一个端点在集合中

6. 独立集点覆盖定理

图 $G = (V, E)$ 中, 点集 $C \subseteq V$ 是点覆盖, 当且仅当它的补集 $V \setminus C$ 是独立集.

在二分图中可以用这个定理求出最大独立集.

7. 有向无环图最小路径覆盖

- 为每个顶点 $v \in V$, 分别建立一个入点 v_{in} 和一个出点 v_{out} . 设全体入点和出点的集合分别为 $V_{in} = \{v_{in} \mid v \in V\}$ 和 $V_{out} = \{v_{out} \mid v \in V\}$ 它们分别成为新图的左部和右部.
- 为每条有向边 $(u, v) \in E$, 建立无向边 (u_{out}, v_{in}) . 全体无向边的集合就是 $E' = \{(u_{out}, v_{in}) \mid (u, v) \in E\}$

有向无环图 $G = (V, E)$ 的最小路径覆盖与相应的二分图 $G' = (V_{in}, V_{out}, E')$ 的最大匹配的大小之和等于顶点数量.

首先, 不交的路径和二分图的一个匹配是一一对应的.

所有失配的节点是 DAG 路径上的起点. 个数等于路径条数.

最小路径覆盖问题是指, 在一张有向图中, 选择最少数量的简单路径, 使得所有顶点都恰好出现在一条路径中.

8. 最小路径可重点覆盖

在最小路径可重点覆盖中, 若两条路径 $(\dots \rightarrow u \rightarrow p \rightarrow v \rightarrow \dots)$ 和 $(\dots \rightarrow x \rightarrow p \rightarrow y \rightarrow \dots)$ 在点 p 相交, 则我们在原图中增加一条边 (x, y) , 让第二条路径通过这条有向边, 就可以避免重复覆盖点 p .

进一步的, 如果我们把原图中所有间接连通的点对 x, y 直接连上有向边 (x, y) , 那么任何 "有路径相交的点覆盖" 一定能转化成 "没有路径相交的点覆盖".

综上所述, 有向无环图 G 的最小路径可重复点覆盖, 等价于先对有向图传递闭包, 得到有向无环图 G' , 再在 G' 上求一般的 (路径不可相交) 最小路径点覆盖.

最小路径可重点覆盖是指, 给定一张 DAG, 要求用尽量少的可相交的简单路径, 覆盖有向无环图的所有顶点 (也就是每个顶点可以覆盖多次).

9. 有向无环图的最大独立集

根据 Dilworth 定理, 最大独立集等于最小路径可重点覆盖

这本来是一个偏序集上的定理, 任意的有向无环图传递闭包后都相当于偏序集

构造方案: 拆点二分图的 $x_1 \rightarrow x_{2+n} \rightarrow x_2 \rightarrow x_{3+n} \dots x_{k+n}$ 对应的是原 Dag 中的一条路径 $x_1 \rightarrow x_2 \rightarrow x_3 \dots x_k$, x_1 即为路径起点, x_k 即为终点. 先将所有的终点放入独立集集合 H , 再将与 H 连有向边的点加入集合 N , 由于已经传递闭包, 当 $H \cap N = \emptyset$ 时, 显然在原图中没有路径相连.

否则, 将 $H - H \cap N$ 的元素 y 向路径起点跳, 设使他往上的是 x . 论证 y 不会跳超过其路径起点.

1. x 是路径的终点, 如果 y 是路径的起点, 那么 $x \rightarrow y$ 可以组成更小的路径覆盖.
2. x 是路径的一部分, 若 x 已经到达现在的位置, 所以他以下的部分必然被接管了, 如果 y 是路径的起点, 那么 $x \rightarrow y$ 可以组成更小的路径覆盖.

所以矛盾

最后 H 集合就是独立集集合

10. 二分图的必须边和可行边

给定一张二分图, 其最大匹配方案不一定是唯一的. 若任何一个最大匹配方案的匹配边都包括 (x, y) , 则称 (x, y) 为二分图匹配的必须边. 若 (x, y) 至少属于一个最大匹配的方案, 则称 (x, y) 为二分图匹配的可行边.

保证完备匹配时

我们先求出任意一组完备匹配的方案, 此时所有节点都是匹配点.

根据定义, (x, y) 是必须边, 当且仅当以下两个条件满足:

1. (x, y) 当前是匹配边.
2. 删除边 (x, y) 后, 不能找到另一条从 x 到 y 的增广路.

(x, y) 是可行边, 当且仅当满足以下两个条件之一:

3. (x, y) 当前是匹配边.
4. (x, y) 是非匹配边, 设当前 x 与 u 匹配, y 与 v 匹配, 连接边 (x, y) 后, 节点 u, v 失去匹配.

必须找到一条从 u 到 v 的增广路, 来保证最大匹配不变.

如果我们把二分图 G 中的“非匹配边”看作从左部到右部的有向边, 把二分图 G 中的“匹配边”看作从右部到左部的有向边, 构成一张新的有向图, 记为 G' , 那么 G 中从 x 到 y 有增广路, 等价于 G' 中存在从 x 到 y 的路径.

考虑必须边的条件, (x, y) 是匹配边, 对应 G' 中有一条从 y 到 x 的有向边. 在此条件下, 若 x 到 y 之间还存在一条路径, 则 x 和 y 处于同一个强连通分量中.

因此, 必须边的判定条件是: (x, y) 当前是二分图 G 的匹配边, 并且 x, y 两点在有向图 G' 中属于不同的 Scc .

类似, 可行边的判定条件是: (x, y) 当前是二分图 G 的匹配边, 或者 x, y 两点在有向图 G' 中属于相同的 Scc .

综上, 非必须可行边的判定条件是, x, y 两点在有向图 G' 中属于相同的 Scc .

不保证完备匹配时

在这种情况下, 必须边, 可行边的第二个条件会出现问题. 设 (x, y) 是非匹配边:

5. x, y 二者之一可能本来就是非匹配点. 不妨设 y 是非匹配点, 此时直接断开 x 原来的匹配边, 连接 (x, y) , 得到的匹配集仍然是最大匹配.
6. 即使当前 x 与 u 匹配, y 与 v 匹配, 连接边 (x, y) 后, 节点 u, v 失去匹配, 我们也不一定要找到从 u 到 v 的增广路, 如果从 u 出发找到另一个非匹配点 z 的增广路, 同样可以得到一组包含 (x, y) 的最大匹配.

所以我们不能像完备匹配中一样, 直接用强连通分量进行判定. 然而, 我们可以借助网络流中的源点和汇点.

若 z 是当前非匹配点, 则 (z, T) 的剩余容量必定为 1. 若 v 当前是匹配点, 则 (v, T) 的剩余容量必定为 0, (T, v) 的剩余容量必定为 1. 换言之, 残量网络中存在路径 $(\dots \rightarrow z \rightarrow T \rightarrow v \dots)$. 若二分图中 u 到 z 有增广路, 则残量网络上 u 能到达 z , 进而到达 v , 即 (u, v) 通过汇点 T 仍在同一个强连通分量中.

因此, 必须边的判定条件是: (x, y) 流量为 1 (不是残量网络), 并且 x, y 两点在残量网络中属于不同的 Scc .

类似, 可行边的判定条件是: (x, y) 流量为 1, 或者 x, y 两点在残量网络中属于相同的 Scc .

11. km 算法 特殊情况 :

1. 通过为右部点添加虚点来保证左部点个数小于等于右部点个数. (是必要的, Km 需要在完备匹配中保证正确)
2. 然后, 我们要将原本不存在的边连成虚边.
3. 对于需要特判无解的题目, 将虚边边权设为 $-Inf$, 如果被迫选了 $-Inf$ 的边就输出无解. (实现上就是先跑 Km , 然后在统计答案时判断有没有虚边)
4. 对于允许非完美匹配的题目, 将虚边边权设为 0 即可.
5. (对于无解, 也可以选择加一个虚点作为退路, 跑一遍 Km , 如果该虚点被匹配则必然无解)
6. 对于保证最大匹配求最大权匹配, 将虚边边权设为 $-Inf$, $-Inf$ 的边在统计答案时删掉.
7. 对于不保证最大匹配求最大权匹配, 将负边权设为零, 连 0 虚边, 统计答案.

2-Sat

模型

有 N 个变量, 每个变量只有两种可能的取值. 在给定 M 个条件, 每个条件都是对两个变量的取值限制. 求解: 是否存在对 N 个变量的合法赋值, 使 M 个条件均得到满足.

设一个变量 $A_i (1 \leq i \leq N)$ 的两种取值分别是 $A_{i,0}$ 和 $A_{i,1}$. 在 $2-Sat$ 问题中, M 个条件都可以转化成统一的形式: 若变量 A_i 赋值为 $A_{i,p}$, 则变量 A_j 必须赋值为 $A_{j,q}$, 其中 $p, q \in \{1, 0\}$.

判定

1. 建立 $2N$ 个节点的有向图, 每个变量 A_i 对应两个节点, 记为 i 和 $i + N$.
2. 考虑每一个条件, 形如, 若变量 A_i 赋值为 $A_{i,p}$, 则变量 A_j 必须赋值为 $A_{j,q}$, 其中 $p, q \in \{1, 0\}$, 从 $i + p * N$ 到 $j + q * N$ 连一条有向边. 注意: 上述条件蕴含了它的逆反命题, 即, 若变量 A_j 赋值为 $A_{j,1-q}$, 则变量 A_i 必须赋值为 $A_{i,1-p}$, 若给出的条件不包含它的逆反命题, 应从 $j + (1 - q) * N$ 到 $i + (1 - p) * N$ 连一条有向边.
3. 用 $Tarjan$ 算法求出当前有向图的所有强连通分量.
4. 若存在 $i \in [1, N]$, 满足 i 和 $i + N$ 属于同一个强连通分量, 则说明若变量 A_i 赋值为 $A_{i,p}$, 则变量 A_i 必须赋值为 $A_{i,1-p}$, 显然不可能.
5. 根据 $Tarjan$ 算法的特性, 编号更小的 Scc 显然处于缩点后拓扑序的末尾, 同时, 在一个 Scc 中, 只要确定一个变量的取值, 其他 Scc 的取值也确定了; 在缩点后的 Dag 中, 如果一个点有出度, 会使出边的点被强制更新, 所以我们应该选择出度为零的点. 综上, 如果 $Belong_i > Belong_{i+N} (i \in [1, N])$, 我们令变量 A_i 赋值成 A_{i+N} .

构图方法

1. 如果有一个变量的取值直接导致无解, 则把这个取值和另一个取值连有向边, 这样如果被迫选择了这个变量, 直接无解.
2. 如果某个变量必须取某个取值, 那么把另一个取值直接和这个取值连有向边.

Tarjan

点双连通分量对边染色

```
static std::function<void(int,int)> Tarjan=[&](int from,int root)->void {
    dfn[from]=low[from]=++tot;
    insta[from]=true;
    for(auto to:To[from]){
        if(!dfn[to]){
            int siz=sta.size();
            Tarjan(to,root);
            low[from]=std::min(low[from],low[to]);
            if(dfn[from]<=low[to]){
                do{
                    auto now=sta.back();sta.pop_back();
                    insta[now.first]=false;
                    insta[now.second]=false;
                }while((int)sta.size()!=siz);
                insta[from]=true;
            }
        }
        else {
            low[from]=std::min(low[from],dfn[to]);
            if(insta[to])sta.push_back({from,to});
        }
    }
};
for(int i=1;i<=n;i++)
    if(!dfn[i])Tarjan(i,i);
```

点双连通分量对点染色

```
static std::function<void(int, int)> Tarjan = [&](int from, int root) -> void {
    Dfn[from] = Low[from] = ++tot;
    S.push_back(from);
    if (from == root && G[from].empty()){
        VDcc[++vdcc].push_back(from);
        return;
    }
    int flag = 0;
    for (int to : G[from]){
        if (!Dfn[to]){
            Tarjan(to, root);
            Low[from] = std::min(Low[from], Low[to]);
            if (Dfn[from] <= Low[to]){
                flag++;
                if (from != root || flag > 1){
                    Cut[from] = true;
                }
            }
            vdcc++;
            int now;
            do{
                now = S.back();
            }while(now != to);
            S.pop_back();
        }
    }
    if (flag > 1)VDcc[++vdcc].push_back(from);
    VDcc[vdcc].push_back(from);
};
```

```

        S.pop_back();
        VDcc[vdcc].push_back(now);
    } while (now != to);
    VDcc[vdcc].push_back(from);
}
}
else{
    Low[from] = std::min(Low[from], Dfn[to]);
}
}
};
std::vector<int> Tree[2 * MaxN];
std::vector<int> Id(MaxN, 0);
int num = 0;
inline void Insert(std::vector<int> adj_list[], int u, int v){
    adj_list[u].push_back(v);
}
inline void Work(){
    num = vdcc;
    for (int i = 1; i <= n_nodes; i++){
        if (Cut[i]){
            Id[i] = ++num;
        }
    }
    for (int i = 1; i <= vdcc; i++){
        for (int v : VDcc[i]){
            if (Cut[v]){
                Insert(Tree, i, Id[v]);
                Insert(Tree, Id[v], i);
            }
            else{
                Belong[v] = i;
            }
        }
    }
}
}
}

```

边双连通分量

```

int Belong[MaxN], edcc;
inline void Dfs(int from){
    Belong[from] = edcc;
    for (register int k = G.Last[from]; k != -1; k = G.Next[k]){
        if (Belong[G.To[k]] || Bridge[k]) continue;
        Dfs(G.To[k]);
    }
}

Graphs<MaxN, MaxN> Tree;

inline void Work(){
    for (register int i = 1; i <= n; i++){
        if (!Belong[i]){

```

```

        edcc++;
        Dfs(i);
    }
}
for(register int i=0;i<G.cntm;i++){
    if(Belong[G.To[i^1]]==Belong[G.To[i]])continue;
    Insert(Tree,Belong[G.To[i^1]],Belong[G.To[i]]);
}
}

```

强连通分量

```

Graphs<MaxN,MaxM>G;
Stack<int,MaxN>S;
vector<int>Scc[MaxN];
int Dfn[MaxN],Low[MaxM],tot,scc;
bool Visit[MaxN];
inline void Tarjan(int from){
    Dfn[from]=Low[from]=++tot;
    S.Push(from);Visit[from]=true;
    for(register int k=G.Last[from];k!=-1;k=G.Next[k]){
        if(!Dfn[G.To[k]]){
            Tarjan(G.To[k]);
            Low[from]=Min(Low[from],Low[G.To[k]]);
        }
        else if(Visit[G.To[k]])
            Low[from]=Min(Low[from],Dfn[G.To[k]]);
    }
    if(Dfn[from]==Low[from]){
        scc++;int now;
        do{
            now=S.Top();S.Pop();Visit[now]=false;
            Belong[now]=scc;Scc[scc].push_back(now);
        }while(from!=now);
    }
}
Graphs<MaxN,MaxN>Tree;
inline void Work(){
    for(register int i=1;i<=n;i++)
        for(register int k=G.Last[i];k!=-1;k=G.Next[k]){
            if(Belong[i]==Belong[G.To[k]])continue;
            Insert(Tree,Belong[i],Belong[G.To[k]]);
        }
}

```

Max Flow

```

namespace Dinic{
    using flowtype=long long;
    using std::vector;

```



```

struct Graphs{
    vector<int>Head, Cur, High;
    struct Edges{
        int to, nxt;
        flowtype flow;
        Edges(int _to=0, int _nxt=-1, flowtype _flow=0):to(_to),nxt(_nxt),flow(_flow){}
    };
    vector<Edges>Er;
    int cnt, tot, src, snk;
    Graphs(){clear();}
    void init(int _n, int _src, int _snk){
        tot=_n; src=_src; snk=_snk;
        Head.resize(_n, -1);
        Cur.resize(_n);
        High.resize(_n);
    }
    void clear(){
        cnt=0;
        Clear(Head); Clear(Er);
    }
    void insert(int u, int v, flowtype f){
        Er.push_back((Edges){v, Head[u], f});
        Head[u]=cnt++;
    }
};

bool Bfs(Graphs&g){
    std::fill(all(g.High), 0);
    std::queue<int>Q;
    Q.push(g.src);
    g.High[g.src] = 1;
    while(!Q.empty()){
        int fr = Q.front(); Q.pop();
        g.Cur[fr] = g.Head[fr];
        gep(k, fr, g.Head, g.Er) {
            int to = g.Er[k].to;
            if(g.Er[k].flow > 0 && g.High[to] == 0) {
                g.High[to] = g.High[fr] + 1;
                if(to == g.snk) return true;
                Q.push(to);
            }
        }
    }
    return false;
}

flowtype Dfs(Graphs&g, int fr, flowtype f) {
    if(fr == g.snk) return f;
    flowtype test, ret = 0;
    for(auto &k = g.Cur[fr]; k != -1; k = g.Er[k].nxt){
        auto to = g.Er[k].to;
        auto flow = g.Er[k].flow;
        if(flow > 0 && g.High[to] == g.High[fr] + 1) {
            test = Dfs(g, to, std::min(flow, f-ret));
            ret += test;
        }
    }
}

```

```

        g.Er[k].flow -= test;
        g.Er[k^1].flow += test;
        if(f == ret) return f;
    }
}
g.High[fr] = 0;
return ret;
}
}
/*Main*/
while(Dinic::Bfs(g)){
    ans+=Dinic::Dfs(g,s,Inf);
}

```

Min Cost Max Flow

Spfa

```

namespace MCMF{
    using flowtype=long long;
    using costtype=long long;
    using std::vector;
    struct Graphs{
        vector<int>Head, Cur;
        vector<costtype>Dist;
        vector<bool>Vis;
        struct Edges{
            int to, nxt;
            flowtype flow;
            costtype cost;
            Edges(int _to=0, int _nxt=-1, flowtype _flow=0, costtype _cost=0):
                to(_to), nxt(_nxt), flow(_flow), cost(_cost){}
        };
        vector<Edges>Er;
        int cnt, tot, src, snk;
        Graphs(){clear();}
        void init(int _n, int _src, int _snk){
            tot=_n; src=_src; snk=_snk;
            Head.resize(_n, -1);
            Cur.resize(_n);
            Dist.resize(_n);
            Vis.resize(_n);
        }
        void clear(){
            cnt=0;
            Clear(Head); Clear(Er);
        }
        void insert(int u, int v, flowtype f, costtype c){
            Er.push_back((Edges){v, Head[u], f, c});
            Head[u]=cnt++;
        }
    }
}

```

```

};
const long long Inf=1e18;
bool Spfa(Graphs&g){
    std::fill(all(g.Dist), Inf);
    std::fill(all(g.Vis), false);
    std::queue<int>Q;
    Q.push(g.src); g.Dist[ g.src ] = 0; g.Vis[ g.src ] = true;
    while(!Q.empty()){
        auto fr=Q.front();Q.pop();
        g.Cur[fr] = g.Head[fr];
        g.Vis[fr] = false;
        gep(k, fr, g.Head, g.Er){
            auto flow = g.Er[k].flow;
            if(! (flow>0) )continue;
            auto to = g.Er[k].to;
            auto cost = g.Er[k].cost;
            if(g.Dist[to] > g.Dist[fr] + cost){
                g.Dist[to] = g.Dist[fr] + cost;
                if(!g.Vis[to]){
                    g.Vis[to] = true;
                    Q.push(to);
                }
            }
        }
    }
    return g.Dist[g.snk] != Inf;
}

flowtype Dfs(Graphs&g, int fr, flowtype f) {
    if(fr == g.snk) return f;
    flowtype test, ret = 0;
    g.Vis[fr] = true;
    for(auto &k = g.Cur[fr]; k != -1; k = g.Er[k].nxt){
        auto to = g.Er[k].to;
        auto flow = g.Er[k].flow;
        auto cost = g.Er[k].cost;
        if(!g.Vis[to] && flow > 0 && g.Dist[to] == g.Dist[fr] + cost) {
            test = Dfs(g, to, std::min(flow, f-ret));
            ret += test;
            g.Er[k].flow -= test;
            g.Er[k^1].flow += test;
            if(f == ret){
                g.Vis[fr] = false;
                return f;
            }
        }
    }
    return ret;
}

}

/*Main*/
while(MCMF::Spfa(g)){
    test=MCMF::Dfs(g,s,Inf);
    ansf+=test;
}

```

```

    ansc+=g.Dist[t]*test;
}

```

Dijkstra (没有负圈的情况下)

```

namespace MCMF{
    using flowtype=long long;
    using costtype=long long;
    using std::vector;
    struct Graphs{
        vector<int>Head;
        vector<costtype>Dist,High;
        vector<bool>Vis;
        struct Edges{
            int to,nxt;
            flowtype flow;
            costtype cost;
            Edges(int _to=0,int _nxt=-1,flowtype _flow=0,costtype _cost=0):
                to(_to),nxt(_nxt),flow(_flow),cost(_cost){}
        };
        vector<Edges>Er;
        int cnt,tot,src,snk;
        Graphs(){clear();}
        void init(int _n,int _src,int _snk){
            tot=_n;src=_src;snk=_snk;
            Head.resize(_n,-1);
            Dist.resize(_n);
            High.resize(_n,0);
            Vis.resize(_n);
        }
        void clear(){
            cnt=0;
            Clear(Head);Clear(Er);Clear(High);
        }
        void insert(int u,int v,flowtype f,costtype c){
            Er.push_back((Edges){v,Head[u],f,c});
            Head[u]=cnt++;
        }
    };
    const long long Inf=1e18;

    inline costtype Calc(Graphs&g,const int&k){
        return g.Er[k].cost+g.High[g.Er[k^1].to]-g.High[g.Er[k].to];
    }
    bool Dijkstra(Graphs&g){
        std::fill(all(g.Dist), Inf);
        std::fill(all(g.Vis), false);

        std::priority_queue<std::pair<costtype,int>>Q;
        Q.push({0,g.src});
        g.Dist[ g.src ] = 0;

        while(!Q.empty()){

```

```

        auto fr=Q.top().second;Q.pop();
        if(g.Vis[fr])continue;
        g.Vis[fr] = true;
        gep(k, fr, g.Head, g.Er){
            auto flow = g.Er[k].flow;
            auto to = g.Er[k].to;
            if(flow&&g.Dist[to] > g.Dist[fr] + Calc(g,k)){
                g.Dist[to] = g.Dist[fr] + Calc(g,k);
                Q.push({-g.Dist[to],to});
            }
        }
    }
    return g.Dist[g.snk] != Inf;
}

flowtype Dfs(Graphs&g,int fr, flowtype f) {
    if(fr == g.snk) return f;
    flowtype test, ret = 0;
    g.Vis[fr] = true;
    for(auto k = g.Head[fr]; k != -1; k = g.Er[k].nxt){
        auto to = g.Er[k].to;
        auto flow = g.Er[k].flow;
        if(!g.Vis[to] && flow > 0 && g.Dist[to] == g.Dist[fr] + Calc(g,k)) {
            test = Dfs(g,to, std::min(flow, f-ret));
            ret += test;
            g.Er[k].flow -= test;
            g.Er[k^1].flow += test;
            if(f == ret){
                g.Vis[fr] = false;
                return f;
            }
        }
    }
    return ret;
}

}

/*Main*/
while(MCMF::Dijkstra(g)){
    std::fill(all(g.Vis), false);
    test=MCMF::Dfs(g,s,MCMF::Inf);
    for(int i=0;i<g.tot;i++)
        if(g.Dist[i]!=MCMF::Inf)g.High[i]+=g.Dist[i];
    ansf+=test;
    ansc+=g.High[t]*test;
}

```

Flow Tricky

特定流量

如果你知道网络的最大流，你可以求出 最小费用特定流

分析最小割

考虑点集之间哪些边被割即可。

如果两条相邻的边都断流了，当作只有某一条边被割即可。

最大权闭合子图

闭合子图

对于一个有向图 $G = (V, E)$ ，点集 $V' \subseteq V$ ，且 $\forall x \in V'$ ， x 的出边指向 V' 集合内的点。点集 V' 即为原图 G 的闭合子图。

问题

给每个点附上一个权值（自然有负数），求原图最大的闭合子图。

我们利用网络流工具，对于原图中存在的边，我们连一条流量为 ∞ 的有向边，对每个点，若点值为正，从源点连一条流量为点权的边，若点权为负，则向汇点连一条流量为点权的绝对值的边。

证明此图的最小割与原答案的和等于所有正权边的点值和：

简单割

如果所有割边都满足其中一个端点是源点或汇点，即为简单割。

1. 可以证明此图最小割为简单割。因为如果不是的话一定有流量为 ∞ 的边被割了，这是矛盾的。
2. 割和闭合子图一一对应，闭合子图中的点和源点 S 与其他点 T 的割即为闭合子图对应的割。

如果闭合图不是简单割，那么说明有一条边容量为 ∞ ，则说明闭合图中有一条出边的终点不在闭合图中，矛盾。

因为简单割不含 ∞ 的边，所以不含有连向另一个集合（除 T ）的点，所以其出边的终点都在简单割内部，符合闭合图定义。

3. 数量关系：

我们知道最小割的值为（源点到其他点的流量 + 闭合子图到汇点的流量）。答案为（源点到闭合子图的流量 - 闭合子图到汇点的流量）。他们的和即为正权点的权值和。

最大密度子图

子图密度

对于一个无向图 $G = (V, E)$ ，选择一个点集 $V' \subseteq V$ ，选择边集 E' ， $e \in E'$ 满足两个端点 $x, y \in V'$ ，这个子图的密度为 $\frac{|E'|}{|V'|}$ 。

直接考虑 0/1 分数规划，二分 T 。

$$\exists \frac{|E'|}{|V'|} \geq T \Leftrightarrow \exists |E'| - T|V'| \geq 0 \Leftrightarrow \exists T|V'| - |E'| \leq 0$$

由于点集选定后，导出子图必然是所有边集中最优的直接考虑点集和导出子图边集的关系。

$$\begin{aligned} |E'| &= \frac{\sum_{v \in V'} \text{Deg}_v - \text{Cnt}_v}{2} T|V'| - |E'| = T|V'| - \frac{\sum_{v \in V'} \text{Deg}_v - \text{Cnt}_v}{2} \\ &= \frac{\sum_{v \in V'} 2T - \text{Deg}_v + \text{Cnt}_v}{2} \end{aligned}$$

Cnt_v 即为点 v 和子图外的点组成的边数，不难发现这个可以用割来求。

对于原图中的边 (x, y) 连一条流量为 1 的双向边，对原图所有的点，从源点连出一条流量为边数 m 的边，连出一条流量为 $2T - Deg_v + m$ 到汇点，这两个 m 是为了防止边流量小于零。

此图的最小割等于

$$\begin{aligned} & C(s, x \in T) + C(s, t) + C(x \in S, y \in T) + C(x \in S, t) \\ &= (\sum_{x \in T} m) + 0 + (\sum_{x \in S} \sum_{y \in T} 1) + \sum_{x \in S} (2T - Deg_x + m) \\ &= nm + \sum_{x \in S} (2T - Deg_x) + (\sum_{x \in S} \sum_{y \in T} 1) \\ &= nm + 2T|V'| - 2|E'| \end{aligned}$$

和原式有关，直接做完。

无源汇上下界可行流

给定无源汇流量网络 G 。询问是否存在一种标定每条边流量的方式，使得每条边流量满足上下界同时每一个点流量平衡。

不妨假设每条边已经流了 $b(u, v)$ 的流量，设其为初始流。同时我们在新图中加入 u 连向 v 的流量为 $c(u, v) - b(u, v)$ 的边。考虑在新图上进行调整。

由于最大流需要满足初始流量平衡条件（最大流可以看成是下界为 0 的上下界最大流），但是构造出来的初始流很有可能不满足初始流量平衡。假设一个点初始流入流量减初始流出流量为 M 。

若 $M = 0$ ，此时流量平衡，不需要附加边。

若 $M > 0$ ，此时入流量过大，需要新建附加源点 S' ， S' 向其连流量为 M 的附加边。

若 $M < 0$ ，此时出流量过大，需要新建附加汇点 T' ，其向 T' 连流量为 $-M$ 的附加边。

如果附加边满流，说明这一个点的流量平衡条件可以满足，否则这个点的流量平衡条件不满足。（因为原图加上附加流之后才会满足原图中的流量平衡。）

在建图完毕之后跑 S' 到 T' 的最大流，若 S' 连出去的边全部满流，则存在可行流，否则不存在。

有源汇上下界可行流

给定有源汇流量网络 G 。询问是否存在一种标定每条边流量的方式，使得每条边流量满足上下界同时除了源点和汇点每一个点流量平衡。

假设源点为 S ，汇点为 T 。

则我们可以加入一条 T 到 S 的上界为 ∞ ，下界为 0 的边转化为无源汇上下界可行流问题。

若有解，则 S 到 T 的可行流流量等于 T 到 S 的附加边的流量。

有源汇上下界最大流

给定有源汇流量网络 G 。询问是否存在一种标定每条边流量的方式，使得每条边流量满足上下界同时除了源点和汇点每一个点流量平衡。如果存在，询问满足标定的最大流量。

我们找到网络上的任意一个可行流。如果找不到解就可以直接结束。

否则我们考虑删去所有附加边之后的残量网络并且在网络上进行调整。

我们在残量网络上再跑一次 S 到 T 的最大流，将可行流流量和最大流流量相加即为答案。

“一个非常易错的问题”

S 到 T 的最大流直接在跑完有源汇上下界可行的残量网络上跑。

千万不可以在原来的流量网络上跑。

有源汇上下界最小流

给定有源汇流量网络 G 。询问是否存在一种标定每条边流量的方式，使得每条边流量满足上下界同时除了源点和汇点每一个点流量平衡。如果存在，询问满足标定的最小流量。

类似的，我们考虑将残量网络中不需要的流退掉。

我们找到网络上的任意一个可行流。如果找不到解就可以直接结束。

否则我们考虑删去所有附加边之后的残量网络。

我们在残量网络上再跑一次 T 到 S 的最大流，将可行流流量减去最大流流量即为答案。

费用流

费用流流程基本与最大流一致

Match

二分图匹配

```
struct Graph {
    struct Edge {
        int to;
        int nxt;
    };
    vector<int> head;
    vector<Edge> edges;
    Graph(int n = 0) { init(n); }
    void init(int n) {
        head.assign(n + 1, -1);
        edges.clear();
    }
    void insert_edge(int u, int v) {
        edges.push_back({v, head[u]});
        head[u] = (int)edges.size() - 1;
    }
};
Graph G;
int n, m, ans;
int Pre[MaxN], Match[MaxN], Mark[MaxN];
void augment(int from) {
    int tmp;
    while (from != -1) {
        tmp = Match[Pre[from]];
        Match[Pre[from]] = from;
        Match[from] = Pre[from];
        from = tmp;
    }
}
bool bfs(int start) {
    queue<int> Q;
    Q.push(start);
    while (!Q.empty()) {
```



```

    int from = Q.front(); Q.pop();
    for (int k = G.head[from]; k != -1; k = G.edges[k].nxt) {
        int to = G.edges[k].to;
        if (Mark[to] == start) continue;
        Mark[to] = start;
        Pre[to] = from;
        if (Match[to] == -1) {
            augment(to);
            return true;
        } else {
            Q.push(Match[to]);
        }
    }
}
return false;
}
void reinit() {
    fill(Match, Match + MaxN, -1);
    fill(Mark, Mark + MaxN, -1);
}
void work() {
    ans = 0;
    for (int i = 1; i <= n; ++i) {
        if (Match[i] == -1)
            ans += bfs(i);
    }
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int k;
    cin >> n >> m >> k;
    G.init(n + m);
    reinit();
    for (int i = 0; i < k; ++i) {
        int x, y;
        cin >> x >> y;
        G.insert_edge(x, y + n); // 左部点编号 1~n, 右部点编号 n+1~n+m
    }
    work();
    cout << ans << "\n";
    for (int i = 1; i <= n; ++i) {
        if (Match[i] == -1 || Match[i] > n + m)
            cout << 0 << ' ';
        else
            cout << Match[i] - n << ' ';
    }
    cout << '\n';
    return 0;
}

```

二分图最大权完美匹配

```
using ll = long long;
const ll Inf = 1e18;
const int MaxV1 = 510 * 2;
const int MaxV2 = 510 * 2;
ll Weight[MaxV1][MaxV2], Expect1[MaxV1], Expect2[MaxV2], Slack[MaxV2];
int Match1[MaxV1], Match2[MaxV2], Pre[MaxV2];
int Visit1[MaxV1], Visit2[MaxV2];
int sizev1, sizev2;
void Augment(int from) {
    int tmp;
    while (from != -1) {
        tmp = Match1[Pre[from]];
        Match1[Pre[from]] = from;
        Match2[from] = Pre[from];
        from = tmp;
    }
}
ll Calc(int from, int to) {
    return Expect1[from] + Expect2[to] - Weight[from][to];
}
bool KuhnMunkres(int start) {
    fill(Slack, Slack + MaxV2, Inf);
    queue<int> Q;
    Q.push(start);
    while (true) {
        while (!Q.empty()) {
            int from = Q.front(); Q.pop();
            Visit1[from] = start;
            for (int to = 1; to <= sizev2; ++to) {
                if (Visit2[to] == start) continue;
                ll gap = Calc(from, to);
                if (gap < Slack[to]) {
                    Slack[to] = gap;
                    Pre[to] = from;
                    if (Slack[to] == 0) {
                        Visit2[to] = start;
                        if (Match2[to] == -1) {
                            Augment(to);
                            return true;
                        } else {
                            Q.push(Match2[to]);
                        }
                    }
                }
            }
        }
    }
    ll delta = Inf;
    for (int i = 1; i <= sizev2; ++i)
        if (Visit2[i] != start)
            delta = min(delta, Slack[i]);
    for (int i = 1; i <= sizev1; ++i)
        if (Visit1[i] == start)
```

```

        Expect1[i] -= delta;
    for (int i = 1; i <= sizev2; ++i) {
        if (Visit2[i] == start)
            Expect2[i] += delta;
        else
            Slack[i] -= delta;
    }
    for (int i = 1; i <= sizev2; ++i)
        if (Visit2[i] != start && Slack[i] == 0) {
            Visit2[i] = start;
            if (Match2[i] == -1) {
                Augment(i);
                return true;
            } else {
                Q.push(Match2[i]);
            }
        }
    }
}

void ReInit() {
    for (int i = 0; i < MaxV1; ++i)
        fill(Weight[i], Weight[i] + MaxV2, 0);
    fill(Match1, Match1 + MaxV1, -1);
    fill(Match2, Match2 + MaxV2, -1);
    fill(Expect1, Expect1 + MaxV1, -Inf);
    fill(Expect2, Expect2 + MaxV2, 0);
}

int main() {
    freopen(".in", "r", stdin);
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ReInit();
    int n, m, k;
    cin >> n >> m >> k;
    sizev1 = n;
    sizev2 = max(m, n);
    for (int i = 1; i <= k; ++i) {
        int x, y;
        ll w;
        cin >> x >> y >> w;
        Weight[x][y] = w;
        Expect1[x] = max(Expect1[x], w);
    }
    for (int i = 1; i <= n; ++i)
        KuhnMunkres(i);
    ll ans = 0;
    for (int i = 1; i <= n; ++i)
        ans += Weight[i][Match1[i]];
    cout << ans << '\n';
    for (int i = 1; i <= n; ++i) {
        if (Match1[i] == -1 || Match1[i] > m || Weight[i][Match1[i]] == 0)
            cout << 0 << ' ';
        else

```

```

        cout << Match1[i] << ' ';
    }
    cout << '\n';
    return 0;
}

```

一般图匹配

```

struct Graph {
    struct Edge { int to, nxt; };
    vector<int> head;
    vector<Edge> edges;
    void init(int n) { head.assign(n + 1, -1); edges.clear(); }
    void add_edge(int u, int v) {
        edges.push_back({v, head[u]});
        head[u] = (int)edges.size() - 1;
    }
} G;

int n, m, ans;
int Dad[MaxN], Visit[MaxN], Pre[MaxN], Match[MaxN], Mark[MaxN];

int GetDad(int x) { return x == Dad[x] ? x : Dad[x] = GetDad(Dad[x]); }

int FindLca(int x, int y) {
    static int tim = 0; tim++;
    while (x != 0) {
        x = GetDad(x); Visit[x] = tim;
        x = Pre[Match[x]];
    }
    while (y != 0) {
        y = GetDad(y);
        if (Visit[y] == tim) return y;
        y = Pre[Match[y]];
    }
    return 0;
}

queue<int> Q;

void Group(int x, int lca) {
    while (x != lca) {
        int y = Match[x], z = Pre[y];
        if (GetDad(z) != lca) Pre[z] = y;
        if (Mark[y] == 2) Q.push(y), Mark[y] = 1;
        if (Mark[z] == 2) Q.push(z), Mark[z] = 1;
        Dad[x] = y; Dad[y] = z; x = z;
    }
}

bool Bfs(int start) {
    for (int i = 1; i <= n; i++) Dad[i] = i, Pre[i] = 0, Mark[i] = 0;
    Mark[start] = 1; while (!Q.empty()) Q.pop(); Q.push(start);
}

```

```

while (!Q.empty()) {
    int from = Q.front(); Q.pop();
    for (int k = G.head[from]; k != -1; k = G.edges[k].nxt) {
        int to = G.edges[k].to;
        int lx = GetDad(from), ly = GetDad(to);
        if (Match[from] == to || lx == ly || Mark[to] == 2) continue;
        if (Mark[to] == 1) {
            int lca = FindLca(from, to);
            if (lx != lca) Pre[from] = to;
            if (ly != lca) Pre[to] = from;
            Group(from, lca); Group(to, lca);
        } else if (Mark[to] == 0) {
            if (Match[to] == 0) {
                int x = from, y = to;
                while (y != 0) {
                    int z = Match[x];
                    Match[y] = x; Match[x] = y;
                    y = z; x = Pre[y];
                }
                return true;
            } else {
                Pre[to] = from;
                Q.push(Match[to]);
                Mark[Match[to]] = 1;
                Mark[to] = 2;
            }
        }
    }
}
return false;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    G.init(n);
    for (int i = 1, x, y; i <= m; i++) {
        cin >> x >> y;
        G.add_edge(x, y);
        G.add_edge(y, x);
    }
    fill(Match, Match + n + 1, 0);
    ans = 0;
    for (int i = 1; i <= n; i++)
        if (Match[i] == 0 && Bfs(i)) ans++;
    cout << ans << '\n';
    for (int i = 1; i <= n; i++) cout << Match[i] << ' ';
    cout << '\n';
    return 0;
}

```

Count Circle Number

```
#include <iostream>
using namespace std;

int n, m, total;
int deg[200001], u[200001], v[200001];
bool vis[200001];

struct Edge {
    int to, nxt;
} edge[200001];

int cntEdge, head[100001];

void addEdge(int u, int v) {
    edge[++cntEdge] = {v, head[u]}, head[u] = cntEdge;
}

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    cin >> n >> m;
    for (int i = 1; i <= m; i++) cin >> u[i] >> v[i], deg[u[i]]++, deg[v[i]]++;
    for (int i = 1; i <= m; i++) {
        if ((deg[u[i]] == deg[v[i]] && u[i] > v[i]) || deg[u[i]] < deg[v[i]])
            swap(u[i], v[i]);
        addEdge(u[i], v[i]);
    }
    for (int u = 1; u <= n; u++) {
        for (int i = head[u]; i; i = edge[i].nxt) vis[edge[i].to] = true;
        for (int i = head[u]; i; i = edge[i].nxt) {
            int v = edge[i].to;
            for (int j = head[v]; j; j = edge[j].nxt) {
                int w = edge[j].to;
                if (vis[w]) total++;
            }
        }
        for (int i = head[u]; i; i = edge[i].nxt) vis[edge[i].to] = false;
    }
    cout << total << '\n';
    return 0;
}
```

```
#include <iostream>
#include <vector>
using namespace std;
int n, m, deg[100001], cnt[100001];
vector<int> E[100001], E1[100001];
long long total;
int main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    cin >> n >> m;
    for (int i = 1; i <= m; i++) {
        int u, v;
```

```

    cin >> u >> v;
    E[u].push_back(v);
    E[v].push_back(u);
    deg[u]++, deg[v]++;
}
for (int u = 1; u <= n; u++)
    for (int v : E[u])
        if (deg[u] > deg[v] || (deg[u] == deg[v] && u > v)) E1[u].push_back(v);
for (int a = 1; a <= n; a++) {
    for (int b : E1[a])
        for (int c : E[b]) {
            if (deg[a] < deg[c] || (deg[a] == deg[c] && a <= c)) continue;
            total += cnt[c]++;
        }
    for (int b : E1[a])
        for (int c : E[b]) cnt[c] = 0;
}
cout << total << '\n';
return 0;
}

```